DEEP DIVE

Team Topologies for a Product Company

Four team types, three interaction modes, and what they look like inside a product company — including how to handle DevOps, test automation, cross-team projects, and team evolution

March 2026 | Engineering Digest | Special Edition

FOREWORD

Here's the thing most Team Topologies guides get wrong for product companies: they assume a large enterprise with dozens of internal teams and a sprawling microservice estate. A CLO business isn't that. It has a consolidated API and microservice architecture that *is* the product. Banks are the customers. That single product platform is the thing being sold, and almost every engineer works on it or directly supports it.

This creates a specific challenge that the book doesn't address head-on: when your "platform" is your product, who owns it? The answer — and this is the most important thing in this issue — is that the product is stream-aligned work. The Team Topologies "platform team" is something different entirely. Getting this distinction wrong leads to confused ownership, misaligned priorities, and teams that don't know whether they're serving bank customers or internal engineers. This issue untangles it.

HOW TO READ THIS ISSUE

2 minutes: Read the executive summary and the key takeaway box at the end

10 minutes: Add "The Core Question" section and the platform team deep dive

30 minutes: Read the full issue, including interaction modes, DevOps, test automation, and practical scenarios

EXECUTIVE SUMMARY

What Team Topologies Is — and Isn't

Team Topologies is a framework for organising software teams around four types and three interaction modes. Created by Matthew Skelton and Manuel Pais (2019, second edition 2025), it's become the default language for engineering team structure. The core idea: your team structure is your architecture. Conway's Law — that organisations produce systems mirroring their communication structures — is a force to harness deliberately, not a bug to work around.

What makes it different is its focus on **cognitive load** as the primary constraint. Not "how do we organise for efficiency?" but "how do we keep teams focused enough that they can hold their domain in their heads?" When teams juggle too many concerns — offer logic alongside Kubernetes troubleshooting alongside bank-specific API quirks — they cut corners, quality drops, delivery slows.

The framework defines four team types — stream-aligned, platform, enabling, and complicated subsystem — and three interaction modes — collaboration, X-as-a-Service, and facilitating. The constraint is deliberate. The value isn't in the labels but in forcing every team to answer: what type are we, and how should we interact with the teams around us?

4	3	80%	5–9
TEAM TYPES	INTERACTION MODES	ORGS WITH PLATFORM TEAMS BY 2026 (GARTNER)	IDEAL TEAM SIZE

THE CORE QUESTION FOR US

The Product Is a Platform. But It's Not a "Platform Team."

This is the single most important distinction in this entire issue, and the one most likely to cause confusion when adopting Team Topologies language without thinking it through.

A CLO business has a consolidated API and microservice architecture that is the core of the product. Banks integrate with it. Merchants interact through it. Transaction data flows through it. It *is* the card-linked offers product. Internally, people might call it "the platform" — but in Team Topologies terms, it is **not** a platform. It's stream-aligned work.

THE DISTINCTION THAT MATTERS

Stream-aligned work delivers value to *external* customers. The consolidated API serves banks and merchants. The teams building and running it are stream-aligned — their customer is the bank, the merchant, the cardholder.

Platform team work delivers value to *internal* engineers. CI/CD pipelines, container orchestration, observability, test environments, developer tooling — things that make stream-aligned teams faster. The customer is the engineer on the team next to you.

Manuel Pais, co-author of Team Topologies, draws the line explicitly: platform teams have internal customers; stream-aligned teams have external customers. The product API has external customers. That makes it stream-aligned.

Why this matters in practice

If the teams working on the core product think of themselves as a "platform team," they'll optimise for the wrong things. Platform teams optimise for developer experience, self-service, and reducing cognitive load for other engineers. Stream-aligned teams optimise for delivering value to customers — shipping features banks are asking for, improving transaction matching accuracy, onboarding merchants faster, meeting bank SLAs.

The priorities, the roadmap inputs, the definition of "done," and the success metrics are fundamentally different. A platform team succeeds when other engineers can move faster. A stream-aligned team succeeds when bank customers get more value. Conflating these leads to a team that doesn't know whether to prioritise the bank's feature request or the internal developer experience improvement — and ends up doing neither well.

How to slice the product into streams

Even though the product runs on a single consolidated API, the *value streams* within it are distinct. Different parts of the product serve different customer needs, change at different rates, and require different domain knowledge. The streams might look something like:

Offer management — creation, configuration, targeting rules, lifecycle, the merchant/advertiser CMS. Changes frequently as new offer types and targeting capabilities are added.

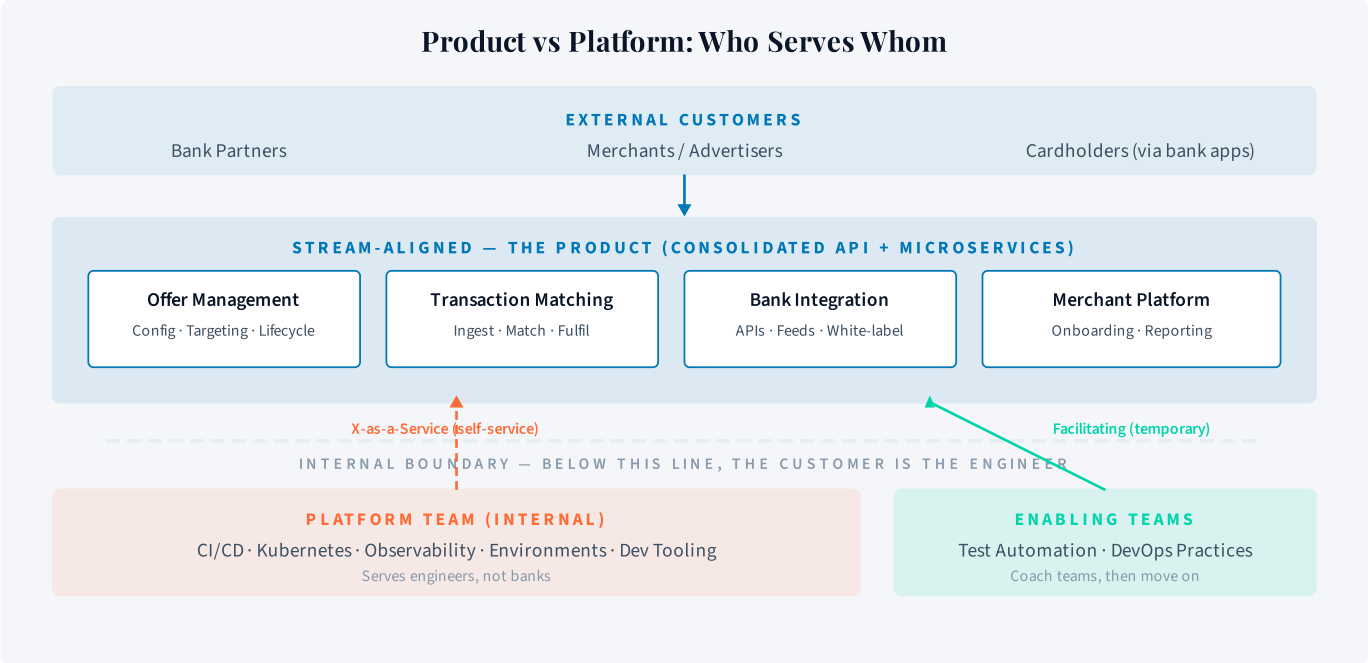
Transaction matching & fulfilment — ingestion of bank transaction data, normalisation, matching against active offers, cashback/reward calculation and settlement. Changes less frequently but requires deep domain expertise.

Bank integration — the APIs, data feeds, and white-label components that bank customers consume. Heavily shaped by per-bank requirements and SLAs.

Merchant onboarding & reporting — the merchant-facing tools, onboarding flows, and performance reporting. A distinct user persona with distinct needs.

Whether these are separate teams or areas of ownership within a smaller number of teams depends on your organisation's size. But the key point holds: all of these are stream-aligned work. They all serve external customers. None of them is a "platform team" in the Team Topologies sense.

Source: Manuel Pais on the distinction — DX Podcast: Platform Teams vs Enabling Teams



THE FRAMEWORK APPLIED

The Four Team Types — Applied to a CLO Business

STREAM-ALIGNED TEAMS

The teams building and running the product — aligned to value streams within the consolidated API

These are the primary teams. They own slices of the product end-to-end: they build, test, deploy, and run their part of the consolidated API in production. Their customers are external — banks, merchants, cardholders. Even though the underlying codebase is a shared API and microservice architecture, the *value streams* are distinct: offer management, transaction matching, bank integration, merchant platform.

In a smaller organisation, one team might own multiple streams. That's fine — what matters is that the ownership is explicit and that the team knows which external customer they're serving. As the organisation grows, streams can be split across teams. The key question: can this team ship a change to their domain without coordinating with another team? If not, the boundaries need rethinking.

What they own: Feature delivery, testing, deployment, and production operations for their area of the product. They carry the pager for their services. They respond to bank escalations about their domain.

PLATFORM TEAM

Internal infrastructure that makes stream-aligned teams faster — not the product itself

The platform team provides internal services that stream-aligned teams consume as self-service. CI/CD pipelines, container orchestration, the observability stack (logging, metrics, alerting), secrets management, environment provisioning, and shared developer tooling. The customer is the engineer on the offer management team, not the bank.

In a smaller organisation, this might not be a dedicated team — it might be a few people who spend part of their time on internal tooling. The key is that the work is recognised as distinct from product work, with a different

customer and different success metrics. The platform succeeds when stream-aligned teams can deploy faster, debug faster, and onboard new engineers faster. It fails when teams are filing tickets and waiting.

This team type is covered in more depth below.

ENABLING TEAMS

Coaches that upskill stream-aligned teams, then move on

Specialists who help stream-aligned teams acquire capabilities they're missing — test automation, DevOps practices, observability skills, security practices — for a bounded period. They don't own software. They don't build features. They pair with developers, help them build the capability, and then rotate to the next team. If an enabling team is still embedded with the same team six months later, something has gone wrong.

Example: The offer management team has poor automated test coverage for the targeting rules engine. A test automation enabling team spends 4–6 weeks helping them design a test strategy, select frameworks, write the first suite of tests, and integrate them into CI. The offer management team now owns their testing. The enabling team moves to the bank integration team next.

COMPLICATED SUBSYSTEM TEAM

Specialists owning a component too complex for a generalist team

Some components require deep specialist knowledge that can't reasonably be expected of a generalist stream-aligned team. In a CLO business, the strongest candidate is the **transaction matching engine** — the algorithms that normalise merchant data from wildly different bank and card network formats and match transactions against active offers. This requires specialist knowledge of merchant identifier formats, fuzzy matching, and the edge cases that arise across Visa, Mastercard, and bank-proprietary feeds.

The subsystem team owns the matching engine and exposes it as an API. Stream-aligned teams call the API — they don't need to understand the normalisation algorithms underneath. If matching accuracy drops, there's one team with the expertise and ownership to fix it.

When to use: Only when the subsystem genuinely requires specialist knowledge. If it can be abstracted into the product's regular stream-aligned work, it should be. Complicated subsystem teams should be rare.

Framework: Team Topologies (IT Revolution, 2019; 2nd edition 2025). Martin Fowler's overview.

Where does UX/UI fit?

This is one of the first questions that comes up when mapping a real organisation to the four types, because most UX/UI teams don't fit neatly into one. The answer depends on what the team actually does — and in practice, a small UX/UI team often operates in multiple modes.

The test is the same one that applies to every team: **who is the customer?**

If the UX/UI team owns a specific product surface end-to-end — say, the merchant portal from design through to production — then they are **stream-aligned**. Their customer is the merchant. They ship features, own the interface in production, and respond to user feedback directly.

But most UX/UI teams in practice operate more like an **enabling team**. They don't own a product stream — they work *across* stream-aligned teams to uplift design quality, establish patterns, build a component library, and coach teams on user research practices. They pair with the offer management team for a few weeks on a redesign, then rotate to the merchant platform team. They don't own the shipped product — the stream-

aligned team does. The interaction mode is facilitating: they help other teams build better interfaces, then move on.

A third dimension exists too: if the team owns and maintains a **shared design system or component library** that other teams consume as self-service, that part of the work is closer to a **platform team** function — providing reusable UI components that reduce cognitive load for stream-aligned teams. The component library has an API (in the broad sense), versioning, documentation, and internal users.

In a smaller organisation, one UX/UI team may do all three. Team Topologies would say that's workable as long as the team and its stakeholders are clear about which mode they're operating in at any given time — and critically, that the enabling and platform work (design system maintenance, coaching on research practices) doesn't get constantly deprioritised in favour of direct product work. The same tension that affects internal platform work versus bank-facing feature work applies here: without explicit time protection, the enabling work always loses.

THE PLATFORM TEAM IN DEPTH

What a Platform Team Does When Your Product Is Already a "Platform"

This is where a CLO context diverges most from the standard Team Topologies guidance. Most examples in the book assume the platform team is the primary infrastructure concern. In a CLO business, the product *is* infrastructure for banks — but the Team Topologies platform team serves a different purpose entirely. It serves the organisation's own engineers.

The purpose: free stream-aligned teams to focus on banks, not Kubernetes

Every hour the offer management team spends debugging a CI pipeline failure, waiting for a test environment, or figuring out Kubernetes networking is an hour not spent on offer targeting improvements that banks are asking for. The platform team's job is to eliminate that extraneous cognitive load. They make the internal machinery invisible so stream-aligned teams can focus entirely on the product domain.

WHAT A PLATFORM TEAM OWNS — CLO CONTEXT

Deployment pipeline: Push code, it builds, runs tests, deploys to staging, and with approval deploys to production. Stream-aligned teams don't configure CI — they just push.

Container orchestration: Kubernetes clusters managed centrally. Teams deploy via simple manifests. They don't manage nodes or cluster upgrades.

Observability: Centralised logging, metrics, alerting. Pre-configured dashboards. If transaction matching latency spikes, the alert fires automatically without the team having built the alerting pipeline.

Environment provisioning: Self-service creation of dev/staging/test environments, including isolated test data that simulates bank feeds without real cardholder data.

Secrets and configuration: Centralised secrets management with self-service rotation. Teams don't manage encryption keys directly.

Developer experience tooling: Local development setup, documentation, internal developer portal, onboarding guides. A new joiner should be able to deploy their first change within a day.

Note what's *not* on this list: the consolidated API itself, the bank-facing endpoints, the offer rules engine, the transaction matching pipeline. Those are product — stream-aligned work. The platform team never touches the product directly. They make it possible for the teams who do to move faster.

> *The difference that platform teams have: internal customers. Product teams — stream-aligned teams as we call them — have external customers, customers outside the organisation.*
>
> — Manuel Pais, Team Topologies co-author

The Thinnest Viable Platform

Skelton and Pais advocate for the "Thinnest Viable Platform" — the smallest set of services that meaningfully reduces cognitive load. For an organisation of this size, the TVP probably starts with the deployment pipeline and observability stack — the two things that create friction for every team. Add services only when stream-aligned teams are consistently spending time on something that could be abstracted away. A bloated internal platform creates its own cognitive load.

When the platform team gets it wrong

ANTI-PATTERN

The Ticket Queue

A stream-aligned team needs a new test environment for a bank onboarding. They file a Jira ticket. The platform team triages and eventually provisions it. Wait time: 4 days. The bank's go-live slips.

THE PATTERN

Self-Service

The team runs a CLI command. A test environment spins up in minutes, pre-configured with tenant isolation and synthetic bank data. The platform team built the tooling; the stream-aligned team consumed it without waiting.

Sources: teamtopologies.com, martinfowler.com, DX Podcast

> *A crucial insight of Team Topologies is that the primary benefit of a platform is to reduce the cognitive load on stream-aligned teams.*
>
> — Martin Fowler

INTERACTION MODES

The Three Ways Teams Work Together

The team types get attention, but the interaction modes are where the framework delivers its sharpest value. Most dysfunction comes from teams interacting in undefined ways. Team Topologies replaces "we need to coordinate better" with three specific modes.

MODE	WHAT IT MEANS	CLO EXAMPLE	DURATION
Collaboration	Two teams work closely on a shared goal. High bandwidth, high cost.	The platform team and bank integration team work together to design automated environment provisioning for multi-bank testing. Neither has the full picture alone.	**Temporary.** Weeks, not months.
X-as-a-Service	One team provides, another consumes with minimal coordination. Clear interface, clear SLAs.	The transaction matching subsystem exposes a matching API. The offer management team calls it to validate transactions. No meetings needed. The platform team provides CI/CD as a service — teams push and deploy.	**Ongoing.** Default for stable capabilities.
Facilitating	One team coaches another to build a capability. They help, not build.	The test automation enabling team pairs with the merchant platform team to build contract tests for the merchant-facing API. After 4 weeks, the team owns it.	**Temporary.** Weeks to a few months.

How modes evolve

A platform capability starts in **collaboration** with an early-adopter team (discover what it should look like), shifts to **X-as-a-Service** once stable (other teams consume independently), and uses **facilitating** to help teams that struggle to adopt. Permanent collaboration signals either a wrong team boundary or an immature service.

DEVOPS & TEST AUTOMATION

Where They Fit — and Where They Don't

DevOps is not a team type

Team Topologies grew from Skelton's earlier DevOps Topologies work, which identified a common anti-pattern: a dedicated "DevOps team" sitting between Dev and Ops, becoming a bottleneck that prevents the collaboration DevOps was supposed to enable.

In the Team Topologies model, DevOps capabilities split across two types:

The platform team owns the tooling: CI/CD pipelines, container orchestration, observability infrastructure,

DEVOPS TOPOLOGIES ORIGIN

Skelton's earlier research identified "Anti-Type B" — a DevOps silo that neither collaborates with Dev nor Ops. Team Topologies evolved from this as a comprehensive org design model.

The key insight: DevOps is a set of practices, not a team. The practices distribute between platform (tooling) and stream-aligned (ownership).

environment provisioning. These are consumed as self-service by stream-aligned teams.

Stream-aligned teams own deployment and production for their part of the product. "You built it, you run it" — but only because the platform is good enough that running it doesn't require deep infrastructure expertise. The offer management team deploys their services and responds to alerts, but they don't manage Kubernetes nodes.

When a stream-aligned team lacks DevOps skills, an **enabling team** bridges the gap: pairing on Terraform, helping set up canary deployments, teaching them to build SLO dashboards. The goal is self-sufficiency — the enabling team moves on once the capability transfers.

Test automation: enabling, not a silo

A centralised QA team is an anti-pattern. It creates a handoff between development and testing that slows flow and separates ownership. Stream-aligned teams should own their own testing.

What if they can't? A **test automation enabling team** rotates across stream-aligned teams on 4–6 week engagements: assessing testing maturity, helping select frameworks, pairing on the first test suites, integrating into CI, establishing patterns for test data management. Then moving on.

The **platform team** provides the shared infrastructure: ephemeral test environments with tenant isolation, synthetic transaction generators that simulate multi-bank feeds without real cardholder data, and CI pipeline integration for parallel test execution.

The enabling team teaches *how* to test. The platform team provides *where* tests run. Both reduce cognitive load, but differently.

Sources: devopstopologies.com, Cloudflight Engineering, Atlassian

CLO-SPECIFIC TESTING CHALLENGES

Card-linked offer testing is harder than typical CRUD apps. You need synthetic transaction feeds simulating multiple banks, merchant IDs in different formats, edge cases in fuzzy matching, and end-to-end flows from transaction ingestion through to cashback fulfilment.

The platform team can provide test data generators and environment isolation. The enabling team helps each team write the actual test cases for their domain.

ENABLING VS PLATFORM FOR TESTING

Enabling: Teaches how — frameworks, patterns, contract testing, test strategy. Temporary.

Platform: Provides where — environments, data, CI, execution. Ongoing service.

PRACTICAL SCENARIOS

Five Situations You'll Recognise

1. A bank onboarding stalls because everyone's busy

Problem: A new bank has signed. Their transaction feed format differs from existing banks. The bank integration team, the platform team, and the offer management team all need to contribute work. Everyone is mid-sprint. The onboarding takes three months.

Diagnosis: Undefined interaction modes. Nobody pre-agreed how cross-team onboarding works.

Fix: Bank onboarding is a *collaboration trigger*. The bank integration team and the platform team enter a time-boxed collaboration to build the adapter and data pipeline together, with shared goals and a shared definition of done. Once the pattern is stable, future onboardings follow the established template (X-as-a-Service) with minimal coordination.

2. Teams can't deploy without the "DevOps team"

Problem: Stream-aligned teams raise tickets for deployments. Two-day wait. Features sit in branches gathering risk.

Diagnosis: The "DevOps team" is a gatekeeper, not a platform.

Fix: Split the work. The *platform function* builds a self-service deployment pipeline. The *enabling function* coaches teams that lack confidence to deploy independently. Within six months, every team deploys without tickets.

3. The centralised QA team is a bottleneck

Problem: Code gets thrown over the wall to QA. Features wait 1–2 weeks for coverage. Bugs return to developers who've context-switched.

Diagnosis: Handoff between development and testing breaks flow and separates quality ownership.

Fix: Dissolve centralised QA over six months. Move QA engineers into stream-aligned teams. Form a test automation enabling team from senior automation engineers to rotate across teams, building capability. Platform team provides shared test infrastructure.

4. Nobody knows who owns the matching engine

Problem: The transaction matching code is maintained by whichever team last touched it. Knowledge of normalisation rules and fuzzy-matching edge cases is spread thinly. When accuracy drops for a bank, nobody owns the fix.

Diagnosis: A complicated subsystem without dedicated ownership.

Fix: Form a *complicated subsystem team* owning the matching engine, normalisation logic, and matching API. Stream-aligned teams call the API; the subsystem team has the expertise to fix accuracy issues. Interaction mode: X-as-a-Service.

5. "Platform" priorities compete with bank feature requests

Problem: Engineers working on internal tooling improvements (CI pipeline speed, observability dashboards) keep getting pulled onto bank-requested feature work, because it's all "the platform." Internal improvements never get finished. Developer experience degrades.

Diagnosis: This is what happens when the product/platform distinction is blurred. Internal tooling work and bank-facing feature work are competing for the same people's time because the organisation treats them as the same thing.

Fix: Explicitly separate the two. Stream-aligned work serves banks — it's prioritised by bank customer needs, commercial commitments, and SLAs. Platform work serves internal engineers — it's prioritised by developer experience metrics, deployment frequency, and cognitive load reduction. Even if it's the same people doing both (which may be necessary at smaller scale), the work needs separate backlogs, separate prioritisation, and explicit time allocation. Without this, bank work will always win, and internal tooling will always lose.

THE TENSION

When Work Crosses Team Boundaries: Projects in a Flow-Based Model

Team Topologies is deliberately anti-project. The framework is built around persistent teams with stable ownership and continuous flow — not temporary project teams assembled for a deliverable and then disbanded. The reasoning is sound: project teams create handoff points, lose context when they dissolve, and nobody owns the code in production once the project is "done."

But the reality is that cross-cutting work happens. A major bank onboarding touches the bank integration team, the offer management team, the matching subsystem, and the platform team. A new product capability — say, real-time offer notification — cuts across multiple streams. A PCI compliance deadline requires coordinated changes everywhere. Pretending these don't need cross-team coordination is as naive as pretending persistent teams don't need to exist.

The question isn't whether cross-team work happens — it's how to handle it without undermining the team ownership model.

The Team Topologies answer: collaboration mode with clear ownership

Cross-team initiatives are handled through *collaboration mode* — a temporary, time-boxed period where teams work closely together on a shared goal. The critical difference from a traditional project: **no team gives up ownership of their domain**. Each team still owns their part of the system. They're collaborating on the *integration points* between their domains, not handing pieces of their work to a project manager.

EXAMPLE: MAJOR BANK ONBOARDING

A new Tier 1 bank has signed. The work touches four teams. Here's how it flows without creating a project team:

Bank integration team owns the bank-specific adapter — the API integration, data feed ingestion, and white-label configuration. This is their stream. They lead the technical relationship with the bank.

Offer management team owns the offer configuration for the new bank — targeting rules, merchant mapping, campaign setup. They work with the bank's commercial team on offer strategy.

Transaction matching subsystem team may need to handle a new merchant ID format or a previously unseen transaction schema. They update the matching engine and expose the capability via their API.

Platform team provisions the tenant infrastructure — data pipeline, environments, monitoring dashboards for the new bank.

Each team works on *their* domain. The cross-team coordination is about *sequencing and integration*, not about reassigning work.

How the EM and PO fit: coordination, not command

This is where engineering managers and product owners play a specific role that Team Topologies doesn't spell out explicitly — but that matters enormously in practice.

The product owner for a cross-team initiative isn't running a project. They're defining the *what* and the *why* at the initiative level — what the bank needs, what the commercial commitments are, what "done" looks like for the bank customer. They work with each stream-aligned team's own PO (or the team directly) to break the initiative into work items that fit each team's backlog and prioritisation. They don't prioritise individual team backlogs — they ensure the teams have enough context to prioritise correctly themselves.

In practice, this means the initiative PO runs a regular cadence (weekly or fortnightly) where representatives from each involved team sync on progress, surface dependencies, and flag blockers. This is not a project standup — it's a coordination point. Each team still has its own sprint cadence, its own backlog, and its own delivery rhythm.

The engineering manager's role is to protect the team ownership model while enabling the cross-team work. This means several things: ensuring the initiative work is properly sized and prioritised in each team's backlog (not bolted on as "also do this"), flagging when a team is being asked to take on initiative work that doesn't fit their current capacity, and — critically — deciding when collaboration mode should end. If the bank integration team and the platform team are still in close collaboration eight weeks into an onboarding, the EM needs to ask whether the integration patterns are mature enough to move to X-as-a-

WHAT THIS LOOKS LIKE IN PRACTICE

Weekly initiative sync (30 mins): Initiative PO + one representative from each involved team. Status, dependencies, blockers. Not a standup — a coordination point.

Team backlogs remain sovereign. Each team prioritises initiative work against their other commitments. The EM helps negotiate when there are conflicts.

Shared Slack channel for the initiative. Async questions, integration testing coordination, quick decisions.

Explicit end date for the collaboration. When does this initiative stop needing cross-team coordination? That's when collaboration mode ends and X-as-a-Service begins.

WHEN THIS BREAKS DOWN

The most common failure: the initiative PO starts acting as a project manager — directly prioritising work in team backlogs, pulling engineers into "war rooms," and creating a shadow delivery structure that undermines team ownership.

The second failure: no explicit coordination at all. Each team picks up their piece independently, nobody tracks the integration points, and the bank go-live slips because Team A shipped their part but Team B didn't know it was urgent.

The balance: enough coordination to keep things aligned, not so much that it replaces team autonomy.

Service for the next bank, or whether the team boundaries need rethinking.

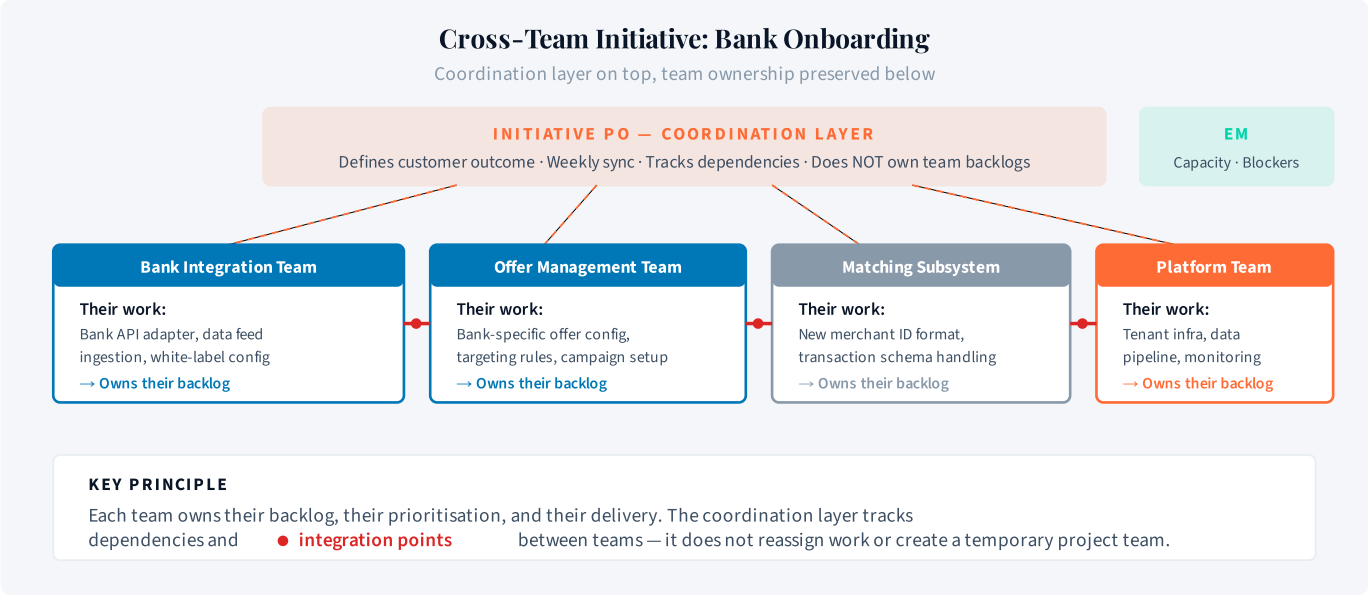
The integration pattern: The initiative PO defines the customer outcome. Each team's PO translates that into domain-specific work items. The EM ensures capacity, removes blockers, and guards team autonomy. Nobody creates a Gantt chart. Nobody reassigns engineers to a project team. The work flows through existing teams with explicit collaboration on the integration points.

Larger initiatives: thin coordination layer, not project management

For bigger efforts — a new product line, a platform migration, a regulatory compliance programme — the coordination layer might be slightly more formal: a dedicated initiative lead (could be an EM, a senior PO, or a principal engineer), a shared roadmap with milestones per team, and regular demo sessions where teams show integrated progress. But the principle holds: each team owns their domain. The coordination layer tracks dependencies and integration points. Nobody builds a temporary project team.

If a cross-team initiative consistently requires so much coordination that it feels like running a project, that's a signal. Either the team boundaries are wrong (the work that needs to be coordinated should be owned by a single team), or the initiative is actually building a new capability that needs its own stream-aligned team.

Sources: Team Topologies key concepts, IT Revolution — interaction modes



THE CORE PRINCIPLE

Cognitive Load: Why This All Matters

The three types

Intrinsic — the inherent complexity of the domain. Understanding offer rules, bank feed formats,

The practical test

Ask each team to list everything they're responsible for. If the list includes their business domain *plus*

matching algorithms. Necessary and unavoidable.

Extraneous — complexity imposed by the environment. Managing Kubernetes, debugging CI failures, provisioning test environments, coordinating deployments across teams. This is what platform and enabling teams exist to reduce.

Germane — effort spent learning and improving. Experimenting with better targeting approaches, improving match accuracy, learning a new bank's requirements. This is where you *want* cognitive effort to go.

infrastructure management, deployment pipelines, test environments, security patching, and cross-team coordination — extraneous load is too high.

When cognitive load exceeds capacity, teams cut corners. Testing gets skipped. Deployment frequency drops. Defect rates rise. The instinct is to add people; Team Topologies argues the better lever is reducing extraneous load through clearer team boundaries, better platform services, and targeted enablement.

TEAM EVOLUTION

When Teams Should Change Size, Shape, or Type

Team Topologies is not a one-time exercise. The authors are explicit: organisational design is never "done" — it must evolve as the business, the technology, and the people change. The signals that a team needs to change are often visible long before anyone acts on them. Knowing what to watch for — and what to do about it — is one of the most practical applications of the framework for engineering managers.

Signals that a stream-aligned team needs to split

The domain has grown beyond one team's cognitive capacity. The bank integration team started by handling three banks. Now there are twelve, each with different feed formats, API requirements, and SLAs. The team can't hold all twelve bank contexts in their heads simultaneously. Changes for Bank A accidentally break Bank B's integration. Deployment frequency has dropped because every change requires regression testing across all banks.

The fix: split by bank cluster or by integration pattern. One team owns the high-volume, real-time feed banks; another owns the batch-file banks with simpler integrations. Each team has a manageable cognitive load. They share the same codebase but own different areas of it.

Delivery cadence has slowed and the team can't explain why. The offer management team used to ship weekly. Now it takes three weeks to get a feature to production. There's no single bottleneck — it's a diffuse slowdown caused by too many things in flight, too many context switches, and too many implicit dependencies within the team's own domain. This is often a sign that the team owns too much.

The fix: identify which parts of the domain change at different rates. Offer configuration and targeting rules change constantly; the underlying offer engine changes rarely. Split along the rate-of-change boundary. The fast-moving team iterates on targeting; the other maintains and evolves the core engine.

Signals that a team needs to merge or absorb work

Two teams are always in collaboration mode. If the bank integration team and the offer management team can never ship independently — every bank change requires an offer change and vice versa — the team boundary is in the wrong place. Permanent collaboration is a sign that what looks like two streams is actually one.

A team is too small to be effective. Dunbar's research, referenced heavily in Team Topologies, suggests a minimum effective team size of around 5 people. Below that, the team lacks the cross-functional skills to own their domain end-to-end — they can't cover development, testing, deployment, and on-call without burning people out. If a team has shrunk to 3 people through attrition or reorg, it either needs to recruit or its domain needs to merge back into a neighbouring team.

Signals that a new team type is needed

Multiple teams are struggling with the same capability gap. If three stream-aligned teams all have poor test automation, and none has the time or expertise to fix it alone, that's the signal for an enabling team. Not a permanent testing function — a temporary coaching team that rotates across the organisation, building capability in each team and then moving on.

A shared component has no clear owner. When a critical piece of the system — the matching engine, the data pipeline, the bank API gateway — is maintained by "whoever touched it last," that's the signal for either a complicated subsystem team (if it requires deep specialist knowledge) or for the platform team to absorb it (if it's infrastructure that multiple teams consume).

The platform team is drowning in tickets. If the platform team has become a bottleneck — long queues, slow turnaround, stream-aligned teams working around them — it might need to grow, or it might need to split. In larger organisations, Skelton and Pais describe "platform groupings" — the platform itself becomes a mini-organisation with its own stream-aligned teams (one owning CI/CD, another owning observability, another owning the data platform), supported by its own enabling team.

> *Team boundaries shouldn't be fixed permanently; they must adapt as products and technologies evolve. The best companies enable teams to negotiate boundary changes directly rather than wait for top-down directives.*
>
> — Team Topologies key concepts

How to evolve without chaos

The most important principle: team changes should be **evolutionary, not revolutionary**. Don't reorganise the entire engineering department on a Monday morning. Instead, identify the strongest signal (the team with the highest cognitive load, the boundary causing the most friction, the capability gap affecting the most teams) and address that one change. Let it settle. Observe the effects. Then address the next one.

Team Topologies provides a useful forcing function for this: every six months, ask each team to map themselves against the four types and three interaction modes. Where are the mismatches? Where are the undefined interactions? Where has a "temporary" collaboration become permanent? Those are your evolution candidates.

Sources: *Team Topologies key concepts*, *IT Revolution — five years*

KEY TAKEAWAYS

→ **The product is not a "platform team."** The consolidated API serves bank customers externally. That makes it stream-aligned work. The Team Topologies platform team serves *internal engineers* — CI/CD, observability, environments, developer tooling. Getting this distinction wrong leads to confused ownership and competing priorities.

→ Stream-aligned teams own slices of the product by value stream (offer management, transaction matching, bank integration, merchant platform) — even if it's a shared codebase. They're responsible for build, test, deploy, and run. Their customer is the bank.

→ The platform team provides internal infrastructure as self-service. If teams file tickets and wait, the platform has failed. The Thinnest Viable Platform — start with CI/CD and observability, add capabilities only when stream-aligned teams are consistently losing time to something that could be abstracted.

→ DevOps is not a team type. Capabilities split between platform (tooling as a service) and stream-aligned teams (deployment and production ownership). Enabling teams bridge the skills gap temporarily.

→ Test automation should be owned by stream-aligned teams. An enabling team helps build the capability; the platform team provides test infrastructure (environments, synthetic data generators, CI integration).

→ The transaction matching engine is a strong candidate for a complicated subsystem team — specialist knowledge of merchant normalisation and fuzzy matching, exposed to stream-aligned teams via API.

→ When product and internal tooling compete for the same people's time, bank work always wins and developer experience always loses. Even at small scale, separate the backlogs, separate the prioritisation, and protect time for internal platform work.

→ **Cross-team initiatives don't need project teams.** Use collaboration mode with a thin coordination layer: an initiative PO defines the customer outcome, each team owns their domain-specific work, and a weekly sync tracks dependencies and integration points. The EM protects team autonomy and decides when collaboration mode should end.

→ **Teams should evolve, not reorganise.** Watch for signals: a domain outgrowing one team's cognitive capacity, permanent collaboration between two teams (they should probably merge), multiple teams struggling with the same capability gap (form an enabling team). Address one change at a time and let it settle.

ON THE RADAR

Coming Up

April 1	AWS Summit Paris
April 19	Claude Haiku 3 model retirement — migrate to Haiku 4.5
April 22	AWS Summit London — expect Bedrock and Trainium announcements

April 29	Zero Day Con, Dublin
April 30	Node.js 20 end of life

RECOMMENDED READING

Sources & Further Reading

Team Topologies — Key Concepts	teamtopologies.com
bliki: Team Topologies	Martin Fowler
Team Topologies: Second Edition (2025)	IT Revolution
Platform Teams vs Enabling Teams — Manuel Pais	DX Podcast
Team Topologies: Five Years of Transforming Organizations	IT Revolution
DevOps Topologies	devopstopologies.com
Enhancing Test Automation with Team Topologies	Cloudflight Engineering
Creditas: Managing Team Cognitive Load for Fast Flow	Team Topologies
Team Topologies and DevOps	Atlassian
The Four Team Types from Team Topologies	IT Revolution

Engineering Digest

March 2026 · Deep Dive · Special Edition

Other issues: Mar 13–20 · Weekly · Mar 11–18 · Weekly · Deep Dive DD-1 · Feb 18 – Mar 18 · Monthly

Next weekly edition: March 24–31, 2026